



ATOMIC

Master Expansion Prompt

Version 4.0 — Agent Architecture & System Design

Directive for AI Coding Agent

FAANG-Grade Engineering Standard

TypeScript · Multi-Agent · Production-Ready

CONFIDENTIAL — INTERNAL USE ONLY

Table of Contents

- §0 Prime Directive
- §1 System Philosophy — The Replit Principle
- §2 Infrastructure Layer
 - 2.1 Internal Event Bus
 - 2.2 Observability Layer
 - 2.3 LLM Abstraction Layer
 - 2.4 Token Budget & Rate Limit Manager
 - 2.5 Prompt Registry
 - 2.6 Output Validation Gates
 - 2.7 Security Model
 - 2.8 Multi-Project Architecture
 - 2.9 Context Window Management
 - 2.10 Testing Architecture
- §3 Blueprint Schema — The Central Artifact
- §4 Agent Workspaces (Spaces)
- §5 Skills System
- §6 Sub-Agent Contract — Pillar Architecture
- §7 Orchestration Protocol
- §8 Feature: Chat Section
- §9 Feature: Artemis — Pre-Pipeline Scoping Agent
- §10 Feature: The Curator — Post-Pipeline Refinement Agent
- §11 Feature: General Mode
- §12 Feature: Blueprint Version History
- §13 Feature: Pipeline & Pillar Improvements
- §14 Settings Page
- §15 Delivery Standard

Prime Directive

■ READ THIS ENTIRE DOCUMENT BEFORE WRITING A SINGLE LINE OF CODE.

You are rebuilding and extending a production application called **Atomic**. Before writing a single line of code, you must complete the following in order:

1. Read and map the **entire existing codebase** — every route, component, hook, service, config, schema, and utility.
2. Document your understanding of the current data flow, state architecture, API layer, and pipeline execution model.
3. Identify every integration point where new features must connect to existing logic.
4. Design the full system architecture before implementation begins.
5. Only then build — incrementally, layer by layer, never leaving the system in a broken intermediate state.

Non-Negotiable Standards

- TypeScript strict mode throughout. **any** is banned unless genuinely unavoidable and justified in a comment.
- No stubs, no **// TODO**, no placeholder logic in any production path.
- Every UI element connects to real logic. Every API call handles loading, error, and edge cases explicitly.
- Streaming must work end to end — no frozen UIs, no dropped tokens, no unhandled backpressure.
- All persistent state survives page refresh and session restart.
- Code must pass review by a senior engineer at a FAANG-level company without embarrassment.
- Every feature must be demonstrably functional — verify each one before considering it done.

SECTION 1

System Philosophy — The Replit Principle

This is the most important architectural directive in this document. It governs every decision you make.

Atomic must function as a **unified, coordinated system** — not a collection of features. The inspiration from Replit is not about specific UI patterns. It is about how a system operates at a fundamental level: every agent, tool, pipeline stage, UI panel, and state object moves in **harmony and coordination**. Every component is aware of system state. Nothing operates in isolation. When one thing moves, the things that depend on it move automatically. The user should never have to manually trigger a consequence the system could have anticipated.

Coordination: Agents share a common state layer via an event bus. When one agent produces output, all components that care are immediately notified and updated without polling or manual refresh.

Reliability: Every agent, sub-agent, and tool has defined failure behavior. The system degrades gracefully — a failing pillar sub-agent does not crash the pipeline.

Robustness: The system handles partial completions, interrupted sessions, network failures, and model errors without losing work. Output is committed to workspace storage before the UI reflects it.

User Experience: The user always has a clear mental model of system state. Progress is visible in real time. Errors are human-readable and actionable. Intervention points are obvious and available.

Trusted and Official Resources — System-Wide Principle

Every agent in this system — Artemis, the Curator, all pillar sub-agents — must prioritize **official, authoritative, and version-specific sources** for any research or knowledge retrieval. No agent treats a blog post or forum answer as equivalent to official documentation. This is baked into every agent's system prompt and tool configuration by default, not an optional setting.

Infrastructure Layer — Build This First

These foundational systems must be built and tested before any agent or feature is implemented on top of them.

2.1 — Internal Event Bus

The event bus is the nervous system of the application. All inter-component communication flows through it. Direct function calls between major components are prohibited.

typescript

```
type AtomicEventType =
  | 'session.created | session.restored'
  | 'artemis.question_sent | artemis.answer_received | artemis.brief_completed'
  | 'pipeline.started | pipeline.completed | pipeline.failed | pipeline.cancelled'
  | 'pillar.queued | pillar.started | pillar.streaming | pillar.completed | pillar.failed |
  pillar.retrying | pillar.improving | pillar.improvement_complete'
  | 'workspace.written | workspace.read | workspace.validated | workspace.validation_failed'
  | 'curator.activated | curator.analysis_started | curator.report_ready | curator.edit_proposed |
  curator.edit_confirmed | curator.edit_applied'
  | 'blueprint.version_created | blueprint.version_restored'
  | 'agent.tool_call_started | agent.tool_call_completed | agent.tool_call_failed'
  | 'token.budget_warning | token.budget_exhausted'
  | 'rate_limit.hit | rate_limit.queued | rate_limit.cleared'
  | 'error.agent | error.system | error.validation'

interface AtomicEvent<T = unknown> {
  id: string
  type: AtomicEventType
  timestamp: string
  traceId: string // follows the entire request chain
  sessionId: string
  payload: T
  metadata?: Record<string, unknown>
}

interface EventBus {
  publish<T>(event: AtomicEvent<T>): void
  subscribe<T>(type: AtomicEventType | AtomicEventType[], handler: (event: AtomicEvent<T>) => void):
  Unsubscribe
  subscribeAll(handler: (event: AtomicEvent) => void): Unsubscribe
  getHistory(filter?: EventBusFilter): AtomicEvent[]
}
```

2.2 — Observability Layer

Every agent action, tool call, state transition, workspace write, and error must emit structured, queryable log entries. This is not optional logging — it is a first-class system component. A developer panel (accessible in dev mode) displays live traces, performance metrics per pillar, token usage, and error history.

typescript

```
interface ObservabilityEntry {
  traceId: string // distributed trace ID across full request chain
  spanId: string // this specific operation
  parentSpanId?: string // which operation triggered this one
  timestamp: string
  level: 'debug' | 'info' | 'warn' | 'error'
  category: 'agent' | 'tool' | 'orchestrator' | 'workspace' | 'ui' | 'system'
  agentId?: string
  pillarId?: string
  sessionId: string
  event: string // human-readable description
  data: Record<string, unknown>
  durationMs?: number
  tokenCount?: number
  model?: string
  error?: SerializedError
}
```

2.3 — Provider-Agnostic LLM Abstraction Layer

All agents use this unified interface. No agent imports an SDK directly. If the provider changes, only the implementation changes — no agent code changes.

typescript

```
interface LLMClient {
  complete(request: LLMRequest): Promise<LLMResponse>
  stream(request: LLMRequest): AsyncIterableIterator<LLMStreamChunk>
  countTokens(content: string | ContentBlock[]): Promise<number>
  isAvailable(): Promise<boolean>
}

interface LLMRequest {
  model: string
  messages: Message[]
  systemPrompt: string
  tools?: ToolDefinition[]
  maxTokens: number
  temperature?: number
  traceId: string // for observability
  budgetTokens?: number // hard stop if approaching token budget
}
```

Graceful Degradation Chain:

1. Attempt with configured model
2. On model unavailability: attempt with global fallback model
3. On rate limit: queue with backpressure, retry with exponential backoff
4. On provider outage: return informative error state, preserve all in-flight work to workspace

2.4 — Token Budget & Rate Limit Manager

When multiple pillar sub-agents run in parallel, all requests are routed through the rate limit manager. No agent calls the LLM client directly without going through this layer.

typescript

```
interface TokenBudgetManager {
  getBudget(sessionId: string): TokenBudget
  consume(sessionId: string, agentId: string, tokens: number): ConsumptionResult
  getUsage(sessionId: string): TokenUsageReport
  isWithinBudget(sessionId: string, estimatedTokens: number): boolean
  onBudgetWarning(handler: (sessionId: string, remaining: number) => void): Unsubscribe
}

interface RateLimitManager {
  enqueue(request: QueuedRequest): Promise<LLMResponse>
  getQueueDepth(): number
  getEstimatedWait(): number
  onRateLimitHit(handler: () => void): Unsubscribe
}
```

2.5 — Prompt Registry

Agent system prompts are versioned, composable artifacts — not hardcoded strings. The composed prompt is validated for coherence before being sent to any agent.

typescript

```
interface PromptTemplate {
  id: string
  version: string
  agentType: AgentType
  basePrompt: string
  injectionPoints: {
    skills: string // where skill modules are injected
    context: string // where dynamic context is injected
    constraints: string // where runtime constraints are injected
  }
}

interface PromptRegistry {
  getTemplate(agentType: AgentType): PromptTemplate
}
```

```
compose(agentType: AgentType, skills: Skill[], context: PromptContext): string
validate(composed: string): ValidationResult
}
```

2.6 — Output Validation Gates

Before any agent commits output to its workspace, it passes through a validation gate. Every workspace has a registered validation gate for its schema. Failed validations emit `workspace.validation_failed` events and trigger the sub-agent's failure strategy.

typescript

```
interface ValidationGate<T> {
  validate(output: unknown): ValidationResult<T>
  onFail: 'retry' | 'flag' | 'block'
  maxRetries: number
}

interface ValidationResult<T> {
  valid: boolean
  parsed?: T
  errors: ValidationError[]
  completenessScore: number // 0-1
  qualityFlags: QualityFlag[]
}
```

2.7 — Security Model

Prompt Injection Defense: All user input that flows into agent prompts must pass through a sanitization layer that strips or escapes known prompt injection vectors.

Workspace Permission Model:

Agent/Role	artemis_workspace	pipeline_workspace	curator_workspace	pillar_workspaces
artemis	WRITE	read	—	—
curator	read	WRITE	WRITE	WRITE
general	read	read	read	read
pipeline	read	WRITE	—	WRITE
system	WRITE	WRITE	WRITE	WRITE

■ The workspace write function enforces this permission table at the data layer. UI that hides edit buttons is defense in depth — actual enforcement lives here.

2.8 — Multi-Project Architecture

Users must be able to have multiple projects. All state, workspaces, blueprints, and version histories are scoped to a `projectId`. Switching projects must cleanly tear down all current agent sessions, flush pending workspace writes, and load the new project state. Zero state bleed between projects.

typescript

```
interface Project {
  id: string
  name: string
  description?: string
  createdAt: string
  lastActiveAt: string
  phase: SystemPhase
  currentBlueprintVersion: number
}

interface ProjectManager {
  createProject(name: string): Promise<Project>
  loadProject(projectId: string): Promise<AtomicSystemState>
  listProjects(): Promise<Project[]>
  deleteProject(projectId: string): Promise<void>
  switchProject(projectId: string): Promise<void>
}
```

2.9 — Context Window Management

This is one of the hardest practical problems in multi-agent LLM systems. The context window manager is called before every agent API call. If the assembled context exceeds the model's context window, it applies the priority ordering below.

Priority ordering (highest to lowest):

1. System prompt + active skills
2. Last N messages from conversation history (always included)
3. Relevant workspace sections (by semantic relevance to current query)
4. Blueprint summary (full blueprint only if within budget)
5. Earlier conversation history (summarized if over budget)
6. Tool call history (summarized)

2.10 — Testing Architecture

A FAANG codebase without tests is not FAANG. LLM systems are notoriously hard to test, but the non-LLM parts are fully unit testable.

- All orchestration state machine transitions (unit tests, no LLM involved)
- Workspace permission enforcement (unit tests)
- Output validation gates for all workspace schemas (unit tests)
- Event bus publish/subscribe behavior (unit tests)

- Token budget and rate limit management (unit tests)
- Context window management and priority ordering (unit tests)
- Blueprint versioning and diff logic (unit tests)
- Agent orchestration flows end to end (integration tests using mock LLM client)
- Critical user flows: Artemis → Pipeline → Curator full cycle (E2E tests)

Build a MockLLMClient that implements LLMClient and returns deterministic, configurable responses. All integration tests use the mock client. Each sub-agent must have a contract test verifying it produces valid output given a valid input and deterministic mock response.

Blueprint Schema — The Central Artifact

The Blueprint is the primary output of the Atomic system. Define it before building anything that produces or consumes it. Every component that reads or writes Blueprint data imports these types. Nothing works with raw or untyped Blueprint data.

typescript

```
interface Blueprint {
  id: string
  projectId: string
  version: number
  overview: {
    projectName: string
    summary: string
    architecture: string
    techStack: TechStack
    designPrinciples: string[]
  }
  pillars: BlueprintPillar[]
  crossCuttingConcerns: {
    security: SecurityPlan
    observability: ObservabilityPlan
    errorHandling: ErrorHandlingPlan
    testing: TestingPlan
    deployment: DeploymentPlan
  }
  dependencies: Dependency[]
  risks: Risk[]
  openQuestions: string[]
  metadata: {
    generatedAt: string
    lastModifiedAt: string
    completenessScore: number
    curatorApproved: boolean
    qualityScore?: QualityScore
  }
}

interface BlueprintPillar {
  id: string
  name: string
  description: string
  status: PillarStatus
}
```

```
content: string
structuredOutput: unknown
dependencies: string[] // other pillar IDs this depends on
providedTo: string[] // other pillar IDs that consume this output
improvementHistory: PillarImprovement[]
subAgentMetadata: SubAgentRunMetadata
}
```

Agent Workspaces (Spaces)

Every agent writes its output to a dedicated, persistent Workspace — a structured output zone that is completely decoupled from the reasoning/thinking stream. The workspace receives only committed, validated, finalized output.

Decoupled — Internal chain-of-thought, tool calls, and intermediate steps are not the workspace. Only committed output lands here.

Persistent — Content survives page refresh, session interruption, and agent re-runs. Stored, not in memory.

Schema-validated — Each workspace has a TypeScript interface. The validation gate enforces it at write time.

Permission-controlled — Write access is enforced at the data layer per the security model in Section 2.7.

Readable by authorized agents — Workspaces are the communication medium between agents. Event bus notifies subscribers on update.

Snapshotted before overwrite — Any workspace update captures the previous state before writing.

Visible to the user — Each workspace has a dedicated UI panel where the user sees committed output in real time.

Agent	Workspace ID	Contents
Artemis	artemis_workspace	Project Brief, Task Breakdown, Requirement Log, Q&A; transcript, confidence score
Pipeline Orchestrator	pipeline_workspace	Full Blueprint, run metadata, pillar dependency graph, overall status
Each Pillar Sub-Agent	pillar_workspace_{id}	Pillar output, tool results, improvement history, run metadata, quality score
Curator	curator_workspace	Refinement Report, ratings, proposed/applied diffs, recommendation history, source citations

Skills System

A **Skill** is a named, composable unit that modifies how an agent thinks and communicates. Skills are *not* tools — tools do things; skills shape how the agent reasons. Skills are stacked and injected in `injectionPriority` order through the Prompt Registry. Multiple skills can be active simultaneously.

```
typescript
interface Skill {
  id: string
  name: string
  description: string
  systemPromptModule: string // injected at the skills injection point
  toolRestrictions?: string[]
  outputFormatOverride?: string
  domainTags: string[]
  compatibleAgents: AgentType[]
  injectionPriority: number // determines order when multiple skills are active
  isBuiltIn: boolean
  createdAt?: string // for custom skills
}
```

Built-in Skills Library:

Skill ID	Description
security-reviewer	Examines every output through a security lens
scalability-auditor	Focuses on load, concurrency, and growth patterns
api-design-expert	Enforces REST/GraphQL/RPC best practices in all output
ux-focused	Shapes output toward user-facing quality and accessibility
strict-typescript	Enforces type safety and TS best practices in all code
performance-optimizer	Identifies performance bottlenecks and optimizations
accessibility-enforcer	WCAG compliance and inclusive design patterns
documentation-writer	Produces clear, complete, structured documentation

Users can also create **custom skills** with their own `systemPromptModule` content via the Settings page.

Sub-Agent Contract — Pillar Architecture

Each pillar in the Atomic pipeline is not a step. Each pillar IS a Sub-Agent. Every pillar sub-agent must conform to this contract exactly.

```
typescript
interface PillarSubAgent {
  id: string
  name: string
  description: string
  dependsOn: string[] // other pillar IDs that must complete first

  config: {
    model: ModelConfig
    tools: ToolConfig[]
    skills: Skill[]
    maxTokens: number
    temperature: number
    onFailure: FailureStrategy
    maxRetries: number
    retryBackoff: BackoffConfig
  }

  workspace: PillarWorkspace
  validationGate: ValidationGate<PillarOutput>

  run(input: PillarInput, context: SystemContext, traceId: string):
  AsyncIterableIterator<LLMStreamChunk>
  improve(feedback: CuratorFeedback, traceId: string): AsyncIterableIterator<LLMStreamChunk>
  retry(reason: RetryReason, traceId: string): AsyncIterableIterator<LLMStreamChunk>
  cancel(): void

  getStatus(): PillarStatus
  getMetrics(): PerformanceMetrics
}
```

Failure Strategy Definitions

Strategy	Behavior
retry	Exponential backoff up to maxRetries, emit pillar.retrying events
skip	Mark as skipped, continue pipeline, emit warning, flag prominently to user
block	Halt pipeline progression at this pillar, require user intervention
fallback	Run simplified prompt, mark output as low-confidence, continue

Each pillar sub-agent must:

- Route all LLM calls through the provider-agnostic LLMClient
- Route all requests through the RateLimitManager
- Check the TokenBudgetManager before each call
- Emit events through the event bus for all state transitions
- Stream output tokens to its workspace in real time as they arrive
- Commit only validated output to its workspace on completion
- Be independently runnable and improvable without touching other pillars
- Use only official, authoritative, version-specific sources for research

Orchestration Protocol

The Pipeline Orchestrator is the central coordination layer. All inter-agent communication goes through it and the event bus. Nothing communicates directly between agents. All state mutations are pure functions through the orchestrator — no component mutates state directly.

Orchestrator responsibilities:

- Owns the canonical AtomicSystemState — the single source of truth
- Receives and validates Artemis's Project Brief before any pipeline trigger
- Builds the pillar dependency graph and sequences execution accordingly
- Manages parallelism: independent pillars run concurrently; dependent pillars wait
- Handles sub-agent failures per their onFailure strategy
- Signals the Curator when the pipeline reaches a reviewable state
- Exposes intervention points: pause, cancel, re-run specific pillar, inject feedback

System State Schema

typescript

```
interface AtomicSystemState {
  sessionId: string
  projectId: string
  phase: 'scoping' | 'pipeline' | 'refinement' | 'complete'

  workspaces: {
    artemis: ArtemisWorkspace
    pipeline: PipelineWorkspace
    curator: CuratorWorkspace
    pillars: Record<string, PillarWorkspace>
  }

  chat: {
    mode: ChatMode
    artemisThread: Message[]
    curatorThread: Message[]
    generalThread: Message[]
  }

  blueprint: Blueprint | null
  blueprintVersions: BlueprintVersion[]
  currentBlueprintVersion: number

  tokenUsage: TokenUsageReport
  settings: AtomicSettings
  errors: SystemError[]
  lastUpdated: string
}
```

```
}
```

Project Brief Schema — Artemis Handoff Contract

typescript

```
interface ProjectBrief {
  projectName: string
  description: string
  problemStatement: string
  targetUsers: string[]

  techStack: {
    frontend?: string[]
    backend?: string[]
    database?: string[]
    infrastructure?: string[]
    preferred?: string[]
    excluded?: string[]
  }

  constraints: {
    timeline?: string
    budget?: string
    teamSize?: string
    technical?: string[]
    business?: string[]
  }

  taskBreakdown: {
    phases: Phase[]
    milestones: Milestone[]
    components: Component[]
    deliverables: Deliverable[]
  }

  successCriteria: string[]
  outOfScope: string[]
  openQuestions: string[]

  completedAt: string
  artemisSessionId: string
  confidenceScore: number // pipeline blocked if below threshold (default 0.75)
}
```

Feature: Chat Section

The Chat Section is a **first-class workspace panel** — not a modal, not a sidebar. Three modes, user-switchable at any time. Each mode maintains an independent conversation thread stored in system state, not component local state.

Mode	Identity	Availability	Blueprint Access
artemis	Pre-pipeline scoping agent	Always	Read (no edit)
curator	Post-pipeline refinement agent	After any pillar completes	Read + Write (exclusive)
general	Blueprint exploration & Q&A;	Always	Read only (enforced at data layer)

Mode Switcher Requirements:

- Persistent in UI state — survives page refresh
- Instant — no loading, no re-initialization on switch
- Each mode has distinct visual identity: name, avatar, color accent
- Switching modes does not interrupt any running agent
- Each mode's conversation thread is fully independent
- Streaming chunks written to UI directly from event bus subscription — no polling

Feature: Artemis — Pre-Pipeline Scoping Agent

Identity: Senior solutions architect and product manager. Methodical, thorough, patient. Never asks more than two questions per message. Never rushes the user. Opens with a single orienting question: *what are you building?*

Questioning Strategy

- Opens with one orienting question, then branches based on answers
- Tracks an internal requirement map of what is known, unknown, and unclear
- When confidenceScore reaches the configured threshold, surfaces the Project Brief in the Artemis workspace for user review
- User can request to proceed with incomplete information — Artemis marks gaps as assumption in the Brief
- The pipeline must not run if confidenceScore is below the configured threshold (default: 0.75)

Task Breakdown

Once the Project Brief is approved, Artemis decomposes the project into structured phases, milestones, components, and deliverables. The breakdown writes to the [artemis_workspace](#) immediately — visible in the workspace panel, not buried in chat. Strategy is configurable:

- Flat list
- Hierarchical phases
- Milestone-based
- Component-based

Tool Use & Sources

All tool calls are visible to the user as expandable traces in the workspace panel. Every tool call Artemis makes uses official, authoritative, version-specific sources.

- Official-source-prioritized web search
- Structured output generation
- Technology research

Settings

- Model selection
- Active skills (multi-select from library)
- Tool access (toggle per tool)
- Task breakdown strategy
- Confidence threshold (default 0.75)

- Tone: technical / business-friendly / hybrid

Feature: The Curator — Post-Pipeline Refinement Agent

Identity: The most capable agent in the system. Senior technical architect, security auditor, and quality engineer simultaneously. Reads with precision. Makes recommendations grounded in authoritative, cited sources. **Never guesses. Never bluffs.**

Only the Curator can edit the Blueprint. This is enforced at the workspace permission layer (Section 2.7), not merely at the UI level.

Refinement Report Schema

typescript

```
interface RefinementReport {
  overallScore: QualityScore

  dimensions: {
    completeness: DimensionScore
    consistency: DimensionScore
    security: DimensionScore
    scalability: DimensionScore
    maintainability: DimensionScore
    bestPractices: DimensionScore
    feasibility: DimensionScore
    observability: DimensionScore
    testability: DimensionScore
  }

  findings: Finding[]
  recommendations: Recommendation[]
  generatedAt: string
  sourcesConsulted: Source[]
  modelUsed: string
  tokenCount: number
}

interface Finding {
  id: string
  severity: 'critical' | 'warning' | 'suggestion'
  pillarId?: string
  description: string
  impact: string
  recommendation: string
  source?: Source
}
```

```

}

interface DimensionScore {
  score: number // 0-100
  summary: string
  keyFindings: string[]
}

```

Official Source Search — Implementation

- **Domain allowlist:** official documentation sites, GitHub repos of named libraries, RFC/spec bodies, standards organizations, official changelogs
- **Source ranking:** allowlisted domains rank above all other results
- **Recency filter:** sources older than configured threshold (default 18 months) are de-prioritized
- **Citation requirement:** every factual claim includes citation with URL, source name, and retrieval date — enforced at prompt level AND verified at output validation
- **Applies everywhere:** in chat, in the Refinement Report, and in edit explanations

Blueprint Editing Rules

- Edits applied as structured diffs — not full rewrites unless necessary
- Every edit creates a new blueprint version before applying
- Curator explains every change before applying it
- User confirms edits (configurable: always-confirm / auto-apply / preview-first)
- Edits applied atomically — partial application is impossible

Pillar-Specific Improvement Loop

The Curator can target any individual pillar sub-agent for a focused improvement pass. It sends structured `CuratorFeedback` to that sub-agent, which runs its `improve()` method. The Curator reads the re-generated output, validates it, and accepts or requests further iteration. This loop continues until satisfied or the user intervenes.

Settings

- Model selection
- Active skills
- Tool access
- Trusted source domain list (add/remove/reorder)
- Refinement depth: surface / deep / comprehensive
- Edit confirmation mode: always-confirm / auto-apply / preview-first
- Source recency threshold (months)

Feature: General Mode

Understand and explore the blueprint without risk of modification. General mode has access to all tools but zero modification capability.

Capabilities

- Full read access to all workspaces, blueprint, and version history
- Ask 'why' questions grounded in the blueprint
- Compare any two blueprint versions
- Generate explanations for different audiences: technical, business, non-technical
- Access all tools (web search, analysis, etc.) for exploration

Hard Constraints — Enforced at Logic Layer, Not UI Layer: • General mode agent system prompt explicitly prohibits modification • The workspace permission table (Section 2.7) enforces read-only at the write function • UI hiding of edit affordances is defense in depth only — the data layer is the real lock

Feature: Blueprint Version History

Every time the blueprint is modified, a new version is created. The original is permanently preserved. Restoring a version always creates a new version — it is never a destructive overwrite.

Version creation triggers:

- Curator applies any edit to the blueprint
- User manually saves a checkpoint
- Pipeline completes a full run
- A pillar improvement pass changes its output

typescript

```
interface BlueprintVersion {
  versionNumber: number // monotonically increasing, never reused
  parentVersion: number | null
  timestamp: string
  author: 'curator' | 'pipeline' | 'user' | 'pillar_subagent'
  authorDetail: string
  changeSummary: string // auto-generated
  changeType: 'full' | 'pillar' | 'checkpoint' | 'restore'
  affectedPillars: string[]
  diff: BlueprintDiff
  snapshot: Blueprint // full snapshot – never delta-only
  integrityHash: string // SHA-256 of the snapshot for corruption detection
}

interface BlueprintDiff {
  added: DiffSection[]
  removed: DiffSection[]
  modified: DiffSection[]
  unchanged: number
}
```

Version History UI Requirements

- Dedicated Version History panel — browsable, searchable, filterable by author/type
- Each entry shows: version number, timestamp, author, change summary, affected pillars
- Click any version to view it in full (read-only preview, does not change active version)
- Restore creates a new version based on the selected old version — never destructive overwrite
- Inline diff view between any two selected versions
- V1 is permanently read-only and never prunable — original is always preserved
- Integrity hash displayed per version — user can verify

Feature: Pipeline & Pillar Improvements

Replit-Grade System Coordination

This is not about aesthetics. When a pillar sub-agent completes, the orchestrator immediately unblocks all dependent sub-agents. The user sees this in real time without any manual action. Everything updates simultaneously. No polling.

- Every pillar card updates status simultaneously via event bus subscription
- Curator's awareness of available pillars updates immediately on completion
- Version history trigger conditions fire automatically
- Chat section context awareness updates in real time
- Independent pillars run concurrently — dependent pillars wait for their dependencies

Live Workspace Output

Tokens stream directly from the sub-agent's `run()` generator into the pillar workspace panel and the UI simultaneously. The user watches the blueprint being built live, section by section, pillar by pillar.

Per-Pillar Status States

Status	Meaning
idle	Not yet queued for this run
queued	Waiting for dependencies to complete
running	Actively generating — streaming to workspace
complete	Output committed to workspace, validated
failed	Encountered an error — see failure strategy
retrying	Retrying after failure with backoff
improving	Curator-triggered improvement pass in progress
paused	User-paused, awaiting resume
skipped	Skipped per failure strategy, flagged to user
low-confidence	Completed via fallback, output marked as low-confidence

Intervention Points

- Pause any running sub-agent
- Cancel and restart a specific sub-agent
- Inject feedback before a sub-agent runs (added to input context)
- Promote a single pillar to the Curator for immediate review without waiting for the full pipeline

Model Configuration

- Each pillar sub-agent has its own model setting, falling back to the pipeline global
- Model list loaded from config — not hardcoded, trivially updatable
- Latest, most capable models are the defaults
- Model list can optionally be fetched from a dynamic endpoint

Settings Page

All settings are persisted and applied at runtime. Changes to active settings take effect on the next agent invocation — no restart required. Organized by agent/component.

Artemis Settings

- Model selection
- Active skills (multi-select)
- Tool access (per tool toggle)
- Task breakdown strategy
- Confidence threshold
- Tone preference

Curator Settings

- Model selection
- Active skills
- Tool access
- Trusted source domain list (add/remove/reorder)
- Refinement depth
- Edit confirmation mode
- Source recency threshold (months)

General Mode Settings

- Model selection
- Active skills
- Tool access

Pipeline Settings

- Global model (fallback for all pillar sub-agents)
- Per-pillar model overrides
- Global tool access
- Maximum parallel sub-agents
- Failure strategy defaults
- Retry configuration

Skills Settings

- Browse built-in skills
- Create custom skills (name, description, systemPromptModule, compatibleAgents)

- Edit custom skills
- Delete custom skills

Version History Settings

- Retention policy: keep all / keep N versions (oldest pruned except V1)

System Settings

- Trusted source domains (global list, inherited by all agents unless overridden)
- Session behavior
- Streaming preferences
- Token budgets
- Developer mode toggle (enables observability panel)

Delivery Standard

Read everything. Design the full system. Build the infrastructure first. Then build the features on top of it. Build it right.

- All infrastructure (Section 2) must be built and tested before any feature is built on top of it
- Every workspace persists to storage — not held in React state only
- All agent sessions maintain coherent context across turns including tool call history
- Streaming is implemented end to end — no simulated streaming, no chunked delays
- The orchestration state machine handles all transitions — no ad hoc state mutations
- Every agent cites its sources when making factual claims
- The system must not lose user work under any failure condition — writes to workspace before UI confirmation
- Skills, settings, and version history survive page refresh and session restart
- Error messages are human-readable, actionable, and never expose raw stack traces to the user
- All critical paths have unit tests. Orchestration flows have integration tests. Critical user journeys have E2E tests.
- The MockLLMClient is built before any integration test is written
- Every feature is verified functional before being considered done

Document
Rating

9.8 /
10

FAANG-Grade & Production-Ready Infrastructure-first architecture
Full agent coordination system